

HW1:

Please write a correct programmer/manager class in C++

```
#include <bits/stdc++.h>
```

```
using namespace std;
```

```
class Employee {
```

```
public:
```

```
    Employee(const char *name, int id);
```

```
    Employee(const Employee &other);
```

```
    Employee& operator=(const Employee &other);
```

```
    ~Employee();
```

```
    const char* getName() const { return _name; }
```

```
private:
```

```
    int _id;
```

```
    char *_name;
```

```
};
```

```
Employee::Employee(const char *name, int id) {
```

```
    _id = id;
```

```
    _name = new char[strlen(name) + 1];
```

```
    strcpy(_name, name);
```

```
}
```

```
Employee::Employee(const Employee &other) {
```

```
    _id = other._id;
```

```
    _name = new char[strlen(other._name) + 1];
```

```
    strcpy(_name, other._name);
```

```
}
```

```
Employee& Employee::operator=(const Employee &other) {
```

```
    if (this == &other) return *this;
```

```
    delete[] _name;
```

```
    _id = other._id;
```

```
    _name = new char[strlen(other._name) + 1];
```

```
    strcpy(_name, other._name);
```

```
    return *this;
```

```
}
```

```
Employee::~~Employee() {
```

```
    delete[] _name;
```

```
}
```

```
int main() {
```

```
    Employee programmer("John", 22);
```

```
    cout << programmer.getName() << endl;
```

```
    Employee manager = programmer;
```

```
    cout << manager.getName() << endl;
```

```
    return 0;
```

```
}
```

HW2:

假設有以下兩個 class，請幫我們實作 Foo 類別的解構子，複製建構子和賦值運算子 (相信我，這題不像看起來的那麼簡單，請考慮 exception-safe 和 memory leak 的各種可能問題)

(參考: <http://isocpp.github.io/CppCoreGuidelines/CppCoreGuidelines#S-ctor>)

```
#include
```

```
class Bar {  
private:  
    std::array not_important_;  
};
```

```
class Foo {  
public:  
    Foo()  
        : bar_{new Bar{}} {}  
  
    // Rule of three!  
    Foo(const Foo& rhs) {  
        bar_ = rhs.bar_ ? new Bar(*rhs.bar_) : nullptr;  
    }
```

```
    Foo&  
    operator=(const Foo& rhs) {  
        if (this == &rhs) {  
            return *this;  
        }  
        Foo tmp(rhs);  
        std::swap(bar_, tmp.bar_);  
        return *this;  
    }
```

```
    ~Foo() {  
        delete bar_;  
    }
```

```
private:  
    // XXX: Well, this is not doing well in practical...  
    // but hey, this is just some exercise. Please use  
    // std::unique_ptr or std::shared_ptr instead.  
    //  
    // The Foo object will "own" this Bar pointer.
```

```
Bar* bar_{nullptr};  
};
```